Async credit processing service

Architecture

FastAPI accepts requests, RabbitMQ holds tasks, and a Celery worker applies credits in MongoDB. A separate dispatcher moves saved work into the queue. I kept the request path independent of RabbitMQ so a broker outage does not hold up API responses.

Request flow

1. `POST /credits` validates the user and amount. One transaction saves the credit and sets the user to `processing`, then the API returns **202 Accepted** with a task ID.
2. The dispatcher claims a saved credit, publishes its ID, and records RabbitMQ's confirmation. Confirmed messages stay in the queue without being republished just because they are waiting.
3. The worker checks the credit, increments the balance, and marks it `processed` in one transaction. Repeating a completed task leaves the balance alone.
4. GET endpoints expose the credit outcome and user balance. A user stays `processing` while any accepted credits remain pending.

I kept two collections:

- **Users:** a unique user ID, Decimal128 balance, status, and pending/failed counts.
- **Credits:** an ID, unique idempotency key, user ID, amount, status, timestamps, dispatch state/lease, and publication and execution counters.

An index on status, dispatch state, and next dispatch time supports the outbox scan.

Key design decisions

The separate credit record answers a question the balance cannot: has this operation already been applied? Updating both records together makes a retry safe even if the worker crashes after committing. A repeated `Idempotency-Key` returns the original credit; changed request data returns a conflict.

Amounts use `Decimal` and `Decimal128`, with strings in JSON responses to preserve decimal values. The assumption is one currency with cent precision. Unknown users are created. Existing numeric balances and missing counters are supported, although converting an old double cannot recover precision already lost.

Implementation approach

The credit record also serves as a transactional outbox: accepting a request saves the obligation to publish it. Publishing only inside the request would leave a crash window between saving and queuing. A flag on the user would be less useful when several credits overlap.

Unconfirmed publication backs off from 30 seconds to a maximum of five minutes. A lease token prevents a late publisher from overwriting a retry already scheduled by a fast worker. Duplicates remain possible if publication succeeds but saving its confirmation fails; the worker's transaction handles them.

Actual execution errors have a separate budget. Unexpected exceptions and non-transient database errors retry with increasing delays, then pause as `failed` after five recorded failures. Waiting in a queue never counts as a failure. Infrastructure and uncertain-commit errors do not consume that budget. If MongoDB cannot save retry intent, Celery retries through RabbitMQ; if that also fails, the original delivery is requeued.

Each worker process lazily reuses its own MongoDB client. Docker Compose configures the services and MongoDB replica set; the README contains run and test commands. Operators can inspect pending work and explicitly requeue failed credits after fixing the cause.

Testing strategy

The tests concentrate on cases that could change money or lose accepted work:

- Duplicate tasks, overlapping credits, and rollback between the balance update and completion record.
- Seeded integer/double balances and user documents without counters.
- Publication backoff, queue backlog, stale lease tokens, and a worker retry racing publisher confirmation.
- Repeated programming errors, transient database failures, and an error reported after a successful commit.
- CLI recovery, dispatcher shutdown, readiness errors, and a complete HTTP-to-worker flow with real dependencies.

The latest results are in `docs/VERIFICATION.md`. Live stop-and-restart checks cover the worker and broker; an OS-level kill at the exact commit boundary remains untested.

For a real launch, I would add account authorization, managed secrets, tested restores, and reconciliation. Confirmed work assumes RabbitMQ retains its persistent data; catastrophic broker data loss needs reconciliation of sent-but-unprocessed credits. Load testing, admission limits, and database failover testing are further work.